Code walkthroughs

Fifteen code blocks that carry most of StrataKV's ideas, each explained line by line with the closest Python and an interview angle. They are teaching versions; after each phase, compare them with the real code in `docs/learning/`. Read the C++ primer tab first if any syntax is unfamiliar.

Durability (P1 and P3)

1. [C++] WAL append and recovery (P1)

```
// Append one WriteBatch as one record. It is acknowledged only after Sync()
returns.
Status WalWriter::AddRecord(std::span<const uint8_t> payload) {
  uint8_t header[9];
  const uint8_t type = kFullRecord;
  uint32_t crc = crc32c::Extend(crc32c::Value(&type, 1), payload.data(),
payload.size());
  EncodeFixed32(header, crc);
  EncodeFixed32(header + 4, static_cast<uint32_t>(payload.size()));
  header[8] = type;
  if (auto s = file_->Append({header, 9}); !s.ok()) return s;
  if (auto s = file_->Append(payload); !s.ok()) return s;
  return sync_every_write_ ? file_->Sync() : Status::OK();
}

// Replay until the first incomplete or corrupt record, then cut the file
there.
Status WalReader::Replay(const std::function<void(std::span<const uint8_t>)>&
apply) {
  uint64_t offset = 0;
  while (true) {
    uint8_t header[9];
    if (!file_->ReadAt(offset, header, 9)) break;                // torn
header
    uint32_t crc = DecodeFixed32(header);
    uint32_t len = DecodeFixed32(header + 4);
    if (len > kMaxRecordSize) break;                             // garbage
length
    std::vector<uint8_t> buf(1 + len);
```

```
    buf[0] = header[8];
    if (!file_->ReadAt(offset + 9, buf.data() + 1, len)) break;   // torn
payload
    if (crc32c::Value(buf.data(), buf.size()) != crc) break;      // torn or
corrupt
    apply({buf.data() + 1, len});
    offset += 9 + len;
  }
  return file_->Truncate(offset);
}
```

Line by line, `AddRecord`:

- `Status WalWriter::AddRecord(...)`: defines the `AddRecord` method of class `WalWriter`, outside the class body (the class itself is declared in the header). It returns a `Status` instead of throwing.
- `std::span<const uint8_t> payload`: a read-only view of the caller's bytes. Nothing is copied, like passing a `memoryview`.
- `uint8_t header[9];`: a fixed 9-byte array on the stack: 4 bytes of checksum, 4 of length, 1 of type. Python: `bytearray(9)`.
- `const uint8_t type = kFullRecord;`: a constant. The `k` prefix is a common naming convention for constants.
- `crc32c::Extend(crc32c::Value(&type, 1), payload.data(), payload.size())`: checksum the type byte, then continue the same checksum over the payload. `&type` passes the address of `type`, because the function takes a pointer plus a length.
- `EncodeFixed32(header, crc)`: writes the 32-bit number into bytes 0 to 3, least significant byte first. Python: `struct.pack_into("<I", header, 0, crc)`.
- `header + 4`: **pointer arithmetic**, meaning "the address 4 bytes into `header`." Python: offset `4` in `pack_into`.
- `if (auto s = file_->Append({header, 9}); !s.ok()) return s;`: append the header; if it fails, return the error immediately. `file_` is a member variable (the trailing underscore is a convention for members); `->` calls a method through a pointer; `{header, 9}` builds a span on the fly.
- `return sync_every_write_ ? file_->Sync() : Status::OK();`: the `?:` ternary is Python's `a if cond else b`. `Sync()` is the `fsync`; only after it returns is the write durable and acknowledged.

Line by line, `Replay`:

- `const std::function<void(std::span<const uint8_t>)>& apply`: a callback that receives each record's payload, like a `Callable[[memoryview], None]` parameter.
- `while (true) { ... break; }`: loop until something looks wrong; every `break` means "stop here, everything before this point is good."
- `ReadAt(...)` returning `false`: fewer bytes were available than asked for, meaning the file ends mid-record.
- `if (len > kMaxRecordSize) break;`: a garbage length from a torn header could be billions; without this check the next line would try to allocate gigabytes.
- `std::vector<uint8_t> buf(1 + len);`: a heap buffer that owns `1 + len` bytes and frees itself at the end of the loop iteration (RAII).
- `buf[0] = header[8];`: put the type byte back in front so the checksum covers exactly what the writer checksummed.
- `buf.data() + 1`: pointer to the second byte, where the payload goes.
- `apply({buf.data() + 1, len})`: hand the payload to the callback, which re-inserts its operations into the memtable.
- `file_->Truncate(offset)`: cut off the partial record so the next append starts on a clean boundary.

Python equivalent of `Replay`:

```
def replay(f, apply):
    offset = 0
    while True:
        header = f.pread(9, offset)
        if len(header) < 9:
            break
        crc, length = struct.unpack_from("<II", header)
        body = header[8:9] + f.pread(length, offset + 9)
        if len(body) < 1 + length or crc32c(body) != crc:
            break
        apply(body[1:])
        offset += 9 + length
    f.truncate(offset)
```

Interview angle: "Why is truncating safe?" The torn record's `fsync` never returned, so no caller was promised it. "What if the bad record is in the middle?" Then valid records follow it, which means real corruption; the production version must report `Corruption` instead of silently discarding acknowledged data.

2. [C++] Atomic file replacement (P3)

```
// After a crash, CURRENT holds either the old name or the new one, never
half of either.
Status SetCurrentFile(Env& env, const std::string& dir, const std::string&
manifest) {
  const std::string tmp = dir + "/CURRENT.tmp";
  if (auto s = env.WriteFile(tmp, manifest + "\n"); !s.ok()) return s;
  if (auto s = env.SyncFile(tmp); !s.ok()) return s;                  // 1.
contents on disk
  if (auto s = env.Rename(tmp, dir + "/CURRENT"); !s.ok()) return s;  // 2.
atomic name swap
  return env.SyncDir(dir);                                            // 3.
the rename on disk
}
```

Line by line:

- `Env& env`: a **reference** to the environment object (file-system wrapper). The function uses the caller's `Env` directly, without copying it; unlike a pointer, a reference can't be null.
- `const std::string& dir`: a read-only reference, the standard way to pass strings into C++ functions without copying.
- `WriteFile(tmp, ...)`: write the new contents to a temporary file, never to `CURRENT` itself, so a crash mid-write can only damage the temp file.
- `SyncFile(tmp)`: force the temp file's bytes onto disk *before* it gets the real name.
- `Rename(tmp, CURRENT)`: on POSIX systems, rename replaces the target in one step. Anyone opening `CURRENT` sees the whole old file or the whole new file.
- `SyncDir(dir)`: a file's name lives in its directory, which is its own piece of data on disk. Syncing the directory makes the rename itself durable.

Python equivalent:

```
def set_current(dir, manifest):
    tmp = os.path.join(dir, "CURRENT.tmp")
    with open(tmp, "w") as f:
        f.write(manifest + "\n")
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, os.path.join(dir, "CURRENT"))
    dfd = os.open(dir, os.O_RDONLY)
    os.fsync(dfd)
    os.close(dfd)
```

Interview angle: this four-step dance (write temp, sync, rename, sync directory) is how databases, package managers, and editors save files safely. Skipping step 1 can leave an empty `CURRENT` after a crash; skipping step 3 can lose the rename.

Data structures (P1 and P2)

3. [C++] Skip list insert with memory ordering (P1)

```
// One writer at a time (it holds the write lock); any number of readers with
no lock.
void SkipList::Insert(std::string key, std::string value) {
  Node* prev[kMaxHeight];
  Node* x = head_;
  const int cur = max_height_.load(std::memory_order_relaxed);
  for (int level = cur - 1; level >= 0; --level) {
    Node* next = x->next[level].load(std::memory_order_acquire);
    while (next != nullptr && cmp_(next->key, key) < 0) {
      x = next;
      next = x->next[level].load(std::memory_order_acquire);
    }
    prev[level] = x;
  }
  const int height = RandomHeight();          // 1 + geometric(p = 1/4),
capped
  if (height > cur) {
    for (int l = cur; l < height; ++l) prev[l] = head_;
    max_height_.store(height, std::memory_order_relaxed);
  }
  Node* node = arena_.New<Node>(std::move(key), std::move(value), height);
  for (int l = 0; l < height; ++l) {
```

```
    // Set the new node's own pointer first; no reader can see the node yet.
    node->next[l].store(prev[l]->next[l].load(std::memory_order_relaxed),
                        std::memory_order_relaxed);
    // Publish. Release means any reader that sees `node` also sees its key,
value, next.
    prev[l]->next[l].store(node, std::memory_order_release);
  }
}
```

What a skip list is: a sorted linked list where each node also has a random number of "express lane" pointers that skip ahead. Searching starts in the highest lane and drops down a level whenever the next node would overshoot, giving roughly O(log n) search, like a balanced tree, but much simpler to make safe for concurrent readers.

Line by line:

- `std::string key, std::string value` taken **by value**: the function gets its own copies (or the caller's moved-in strings), which it then moves into the node.
- `Node* prev[kMaxHeight];` : an array of pointers recording, at each level, the last node before the insertion point. These are the nodes whose pointers will be rewired.
- `max_height_.load(std::memory_order_relaxed)` : read the current tallest level. Relaxed is fine because only the single writer changes it.
- The `for` / `while` pair: the search. At each level, walk right while the next key is smaller, then drop down a level.
- `x->next[level].load(std::memory_order_acquire)` : read a pointer with acquire ordering. Readers do the same thing, which is what makes lock-free reading safe.
- `cmp_(next->key, key) < 0` : the comparator returns negative, zero, or positive, like Python 2's old `cmp()` .
- `RandomHeight()` : each node gets height 1, then keeps a 1-in-4 chance of growing one more level. Most nodes are short; a few are tall express lanes.
- `arena_.New<Node>(...)` : allocate the node in the arena. `std::move(key)` hands over the string's buffer instead of copying it.
- **Publishing order:** first set the new node's forward pointer (nobody can see the node yet, so relaxed is fine), then swing the predecessor's pointer to the new node with a **release** store. A reader that follows that pointer is guaranteed to see the completed node.

Python equivalent of the search (single-threaded, no atomics needed thanks to the GIL):

```
def find_prev(head, key, height):
    prev, x = [None] * height, head
    for level in reversed(range(height)):
        while x.next[level] is not None and x.next[level].key < key:
            x = x.next[level]
        prev[level] = x
    return prev
```

Interview angle: "Why not `std::map` with a lock?" Every read would take the lock, and reads are the hot path. The skip list lets reads proceed with zero locking while a single writer inserts. The cost: correctness depends on memory ordering, which is why every phase runs under ThreadSanitizer.

4. [C++] Internal key ordering (P1)

```
// User key ascending, then (sequence << 8 | type) DESCENDING: newest version
sorts first.
int InternalKeyComparator::Compare(std::string_view a, std::string_view b)
const {
  std::string_view ua = a.substr(0, a.size() - 8), ub = b.substr(0, b.size()
- 8);
  if (int c = ua.compare(ub); c != 0) return c;
  uint64_t ta = DecodeFixed64(a.data() + a.size() - 8);
  uint64_t tb = DecodeFixed64(b.data() + b.size() - 8);
  return ta > tb ? -1 : (ta < tb ? 1 : 0);
}
```

Line by line:

- `std::string_view a, b`: views of internal keys. `substr` on a `string_view` makes another view, with no copying.
- `a.size() - 8`: the last 8 bytes hold the sequence number (7 bytes) and the type (1 byte), so everything before them is the user's key.
- `ua.compare(ub)`: byte-by-byte comparison returning negative, zero, or positive.
- `if (int c = ...; c != 0) return c;`: if the user keys differ, that decides the order.
- `DecodeFixed64(a.data() + a.size() - 8)`: read the trailing 8 bytes as one number. Sequence is in the high bits, type in the low byte.
- `ta > tb ? -1 : ...`: note the reversal. A *larger* sequence number sorts *earlier*, so the newest version comes first.

Python equivalent: `sorted(entries, key=lambda e: (e.user_key, -e.seq_and_type))`.

Interview angle: a `Get` seeks to `(key, current_seq)` and takes the first entry whose user key matches. If that entry is a DELETE, the answer is "not found," even with older PUTs below it. Deletes are just newer entries called tombstones.

5. [C++] Bloom filter (P2)

```
// k probes from one hash via double hashing: h_i = h + i * delta.
// 10 bits per key with k = 7 gives about a 0.8% false-positive rate.
void BloomBuilder::Add(std::string_view key) {
  uint32_t h = Hash(key);
  const uint32_t delta = (h >> 17) | (h << 15);
  for (int i = 0; i < k_; ++i) {
    const uint32_t bit = h % num_bits_;
    bits_[bit / 8] |= static_cast<uint8_t>(1u << (bit % 8));
    h += delta;
  }
}
// MayContain runs the same loop and returns false at the first unset bit.
// "false" is certain; "true" only means "maybe, go read the block".
```

Line by line:

- `uint32_t h = Hash(key);`: one 32-bit hash of the key.
- `(h >> 17) | (h << 15)`: a **bit rotation**: shift right 17, shift left 15, and combine. It produces a second hash-like value from the first, cheaply.
- `h % num_bits_`: pick a bit position in the filter.
- `bits_[bit / 8] |= ... 1u << (bit % 8)`: the filter is stored as bytes, so find the byte (`bit / 8`) and the bit within it (`bit % 8`), then set it with OR. `1u` is an unsigned 1.
- `h += delta;`: the next probe position. `uint32_t` arithmetic wraps around automatically at 2^32.

Python equivalent:

```
def bloom_add(bits: bytearray, key: bytes, k: int, nbits: int) -> None:
    h = hash32(key)
    delta = ((h >> 17) | (h << 15)) & 0xFFFFFFFF
    for _ in range(k):
        b = h % nbits
        bits[b // 8] |= 1 << (b % 8)
        h = (h + delta) & 0xFFFFFFFF   # Python ints never wrap, so mask to
32 bits
```

Interview angle: the best number of probes is k = ln 2 × bits per key, about 7 for 10 bits. More probes than that set too many bits and raise false positives; fewer give each lookup too little evidence.

Compaction and concurrency (P3 and P4)

6. [C++] K-way merge for compaction (P3)

```
void Compact(std::vector<std::unique_ptr<Iterator>>& inputs, SSTableBuilder&
out, bool bottom) {
  auto greater = [&](Iterator* a, Iterator* b) { return icmp_.Compare(a-
>key(), b->key()) > 0; };
  std::priority_queue<Iterator*, std::vector<Iterator*>, decltype(greater)>
heap(greater);
  for (auto& it : inputs) { it->SeekToFirst(); if (it->Valid())
heap.push(it.get()); }

  std::string last_user_key;
  bool have_last = false;
  while (!heap.empty()) {
    Iterator* it = heap.top(); heap.pop();
    std::string_view ikey = it->key();
    std::string_view ukey = UserKey(ikey);
    const bool shadowed = have_last && ukey == last_user_key;   // newer
version already handled
    if (!shadowed) {
      last_user_key.assign(ukey.data(), ukey.size());
      have_last = true;
      const bool tombstone = TypeOf(ikey) == kTypeDeletion;
      if (!(tombstone && bottom)) out.Add(ikey, it->value());
    }
```

```
    it->Next();
    if (it->Valid()) heap.push(it);
  }
}
```

Line by line:

- `std::vector<std::unique_ptr<Iterator>>& inputs`: a list of iterators, one per input SSTable, each owned by a `unique_ptr`. The `&` means the function works on the caller's list.
- `auto greater = [&](...) {...};`: a lambda used as the heap's ordering rule. `[&]` lets it use `icmp_` from the surrounding object.
- `std::priority_queue<..., decltype(greater)> heap(greater);`: C++'s heap is a **max-heap** by default; passing a "greater than" rule flips it into a min-heap, so `top()` is the smallest key. `decltype(greater)` means "the type of that lambda." Python's `heapq` is a min-heap already.
- `heap.push(it.get())`: `.get()` returns the raw pointer inside the `unique_ptr`; the vector still owns the iterator, the heap just points at it.
- `heap.top(); heap.pop();`: in C++ these are separate calls: look at the smallest, then remove it. Python's `heapq.heappop` does both.
- `shadowed`: because of the internal key ordering (snippet 4), the first entry for each user key is the newest. Any later entry with the same user key is outdated and skipped.
- `last_user_key.assign(...)`: copy the key into owned storage, because `ukey` is a view into the iterator's buffer, which changes on `Next()`.
- `if (!(tombstone && bottom))`: keep everything except tombstones at the bottom level.
- `it->Next(); if (it->Valid()) heap.push(it);`: advance the iterator we just consumed and put it back if it has more entries.

Python equivalent:

```
def compact(inputs, out, bottom):
    last = None
    for ikey, value in heapq.merge(*inputs, key=internal_sort_key):
        ukey = user_key(ikey)
        if ukey == last:
            continue                    # older version: drop it
        last = ukey
        if is_tombstone(ikey) and bottom:
            continue                    # nothing below to hide: drop the
delete marker
        out.add(ikey, value)
```

Interview angle: "Why only drop tombstones at the bottom?" A tombstone hides older PUTs in lower levels; drop it higher up and a deleted value comes back. Also: this merge is why compaction has write amplification, since every surviving entry is rewritten.

7. [C++] Version snapshots (P3)

```
Status DBImpl::Get(std::string_view key, std::string* value) {
  std::shared_ptr<MemTable> mem, imm;
  std::shared_ptr<const Version> v;
  uint64_t snapshot;
  {
    std::lock_guard lock(mu_);   // tiny critical section: copy pointers,
nothing else
    mem = mem_; imm = imm_; v = current_; snapshot = last_sequence_;
  }
  // Search mem, then imm, then v's files, with no lock held.
  // Compaction can install a new Version meanwhile; our `v` keeps the old
files alive.
  // When the last shared_ptr to an old Version dies, its unused files become
deletable.
  ...
}
```

Line by line:

- `std::string* value`: an **output parameter**. The function writes the result into the caller's string through the pointer, and uses the return value for the `Status`. Python would just return a tuple.

- `std::shared_ptr<const Version> v;`: a reference-counted pointer to a read-only Version.
- `{ std::lock_guard lock(mu_); ... }`: the extra braces create a scope. The lock is taken when `lock` is created and released automatically at the closing brace (RAII). Python: `with self.mu: ...`.
- `mem = mem_;`: copying a `shared_ptr` increments the reference count. Now this read co-owns the memtable and Version, so neither can be freed while it runs.
- `snapshot = last_sequence_;`: the read ignores any entry newer than this sequence number, giving a consistent point-in-time view.

Python equivalent: every Python object is already reference-counted, so `v = self.current` keeps the Version alive automatically. C++ makes you opt in with `shared_ptr`.

Interview angle: this avoids holding a lock across disk reads, which would block every writer. It's the same idea as MVCC snapshots in Postgres: readers see a fixed version while writers move on.

8. [C++] Group commit (P4)

```
Status DBImpl::Write(WriteBatch* batch) {
  Writer w{batch};
  std::unique_lock lock(mu_);
  writers_.push_back(&w);
  w.cv.wait(lock, [&] { return w.done || &w == writers_.front(); });
  if (w.done) return w.status;                  // a leader already wrote our
batch

  auto [group, last] = BuildBatchGroup();       // leader merges all queued
batches
  const uint64_t seq = last_sequence_ + 1;
  group.SetSequence(seq);
  lock.unlock();                                // new writers can queue
during I/O
  Status s = wal_->AddRecord(group.Contents());  // ONE write, ONE fsync for
everyone
  if (s.ok()) s = InsertInto(group, mem_.get());
  lock.lock();
  last_sequence_ = seq + group.Count() - 1;

  while (true) {                                // wake everyone in the group
```

```
    Writer* f = writers_.front(); writers_.pop_front();
    if (f != &w) { f->status = s; f->done = true; f->cv.notify_one(); }
    if (f == last) break;
  }
  if (!writers_.empty()) writers_.front()->cv.notify_one();  // next leader
  return s;
}
```

Line by line:

- `Writer w{batch};` : a small struct on this thread's stack holding the batch, a `done` flag, a result `status`, and a condition variable `cv`. Other threads reach it through the pointer in the queue.
- `std::unique_lock lock(mu_);` : lock the mutex, using `unique_lock` (not `lock_guard`) because it must be unlocked and relocked mid-function.
- `writers_.push_back(&w);` : join the queue (a `std::deque`, like `collections.deque`).
- `w.cv.wait(lock, [&] { ... });` : release the lock and sleep until notified *and* the condition is true: either someone finished our write, or we reached the front of the queue. On waking, the lock is held again. Python: `cond.wait_for(lambda: ...)`.
- `auto [group, last] = BuildBatchGroup();` : structured binding. The leader merges every queued batch into one, and remembers the last writer it included.
- `lock.unlock();` before the WAL write: the slow `fsync` happens without the lock, so new writers can queue up meanwhile and become the next group.
- The `while` loop at the end: pop each writer in the group, give it the result, mark it done, and wake its thread.
- `writers_.front()->cv.notify_one();` : wake whoever is now first in line to become the next leader.

Python equivalent (shape only):

```
def write(self, batch):
    w = Writer(batch)
    with self.cond:
        self.writers.append(w)
        self.cond.wait_for(lambda: w.done or self.writers[0] is w)
        if w.done:
            return w.status
        group, last = self.build_group()
    status = self.wal.add_record(group)            # one fsync for all
    with self.cond:
        ...pop the group, set results, notify_all()
```

Interview angle: `fsync` costs about the same for 100 bytes as for 100 KB, so sharing one across many writers multiplies throughput under load. Postgres, MySQL, and Kafka all use this idea.

Python and testing (P5)

9. [C++ with pybind11, plus Python] pybind11 binding (P5)

```
namespace py = pybind11;

PYBIND11_MODULE(_stratakv, m) {
  py::register_exception<CorruptionError>(m, "CorruptionError");
  py::class_<stratakv::DB>(m, "DB")
    .def("get", [](stratakv::DB& db, py::bytes key) -> py::object {
        std::string k = key;   // copy out of Python-owned memory while
holding the GIL
        std::string v;
        stratakv::Status s;
        {
          py::gil_scoped_release no_gil;   // let other Python threads run
during disk I/O
          s = db.Get(k, &v);
        }
        if (s.IsNotFound()) return py::none();
        ThrowIfError(s);                  // Status -> Python exception
        return py::bytes(v);
      }, py::arg("key"));
}
```

Line by line:

- `PYBIND11_MODULE(_stratakv, m)`: a macro that defines the Python module `_stratakv`; `m` represents it. The underscore marks it as the private compiled core that the Python wrapper imports.
- `py::register_exception<CorruptionError>(m, "CorruptionError")`: creates a Python exception class; whenever C++ throws `CorruptionError`, Python sees `stratakv.CorruptionError`.
- `py::class_<stratakv::DB>(m, "DB")`: exposes the C++ class as a Python class named `DB`.
- `.def("get", [](...) -> py::object {...}, py::arg("key"))`: adds a method implemented by a lambda. `-> py::object` declares the return type, since it returns either `None` or `bytes`. `py::arg("key")` allows `db.get(key=b"x")`.
- `std::string k = key;`: converts Python `bytes` to a C++ string, copying the data.
- `py::gil_scoped_release no_gil;`: releases the GIL for the rest of its braces (RAII again) and re-acquires it at the closing brace.
- `return py::none();` / `py::bytes(v)`: build Python return values.

What the Python wrapper adds on top:

```
class DB(MutableMapping[bytes, bytes]):
    def __init__(self, core: _stratakv.DB):
        self._core = core
    def __getitem__(self, key: bytes) -> bytes:
        value = self._core.get(key)
        if value is None:
            raise KeyError(key)
        return value
    def __enter__(self): return self
    def __exit__(self, *exc): self._core.close()
```

Interview angle: the GIL lets only one thread run Python at a time, so releasing it during C++ disk work is what makes multi-threaded Python callers actually overlap. The key must be copied first because Python objects may only be touched while holding the GIL.

10. [Python] Model-based testing and the crash fuzzer (P5)

```
class KVMachine(RuleBasedStateMachine):
    keys = st.binary(min_size=1, max_size=16)

    def __init__(self):
        super().__init__()
        self.path = tempfile.mkdtemp()
        self.db = stratakv.DB.open(self.path)
        self.model: dict[bytes, bytes] = {}

    @rule(k=keys, v=st.binary(max_size=256))
    def put(self, k, v):
        self.db.put(k, v)
        self.model[k] = v

    @rule(k=keys)
    def delete(self, k):
        self.db.delete(k)
        self.model.pop(k, None)

    @rule(k=keys)
    def get(self, k):
        assert self.db.get(k) == self.model.get(k)

    @rule()
    def reopen(self):              # every acknowledged write must survive
close/open
        self.db.close()
        self.db = stratakv.DB.open(self.path)

    def teardown(self):
        self.db.close()
        shutil.rmtree(self.path)


def crash_iteration(db_dir: str, rng: random.Random) -> None:
    child = subprocess.Popen([sys.executable, "-m", "tests.py.fuzz_writer",
db_dir],
                             stdout=subprocess.PIPE, text=True)
    time.sleep(rng.uniform(0.05, 2.0))
    child.kill()                   # SIGKILL: no cleanup, no destructors, no
flush
```

```
    acked = dict(line.split() for line in child.stdout.read().splitlines())
    child.wait()
    with stratakv.DB.open(db_dir) as db:
        for k, v in acked.items():
            assert db.get(k.encode()) == v.encode(), f"lost acknowledged
write {k}"
```

Line by line (the Hypothesis parts that may be new):

- `RuleBasedStateMachine`: Hypothesis calls the `@rule` methods in random orders with random arguments, many times, checking assertions as it goes.
- `st.binary(min_size=1, max_size=16)`: a **strategy**, a recipe for generating random values: here, byte strings of length 1 to 16. Short keys make collisions likely, which is good: overwrites and deletes of existing keys get tested.
- `self.model`: a plain dict that behaves the way StrataKV should. Every rule updates both, and `get` checks they agree.
- `reopen`: closes and reopens the database mid-sequence, so recovery code runs constantly, not just in a dedicated test.
- **Shrinking:** when an assertion fails after, say, 300 random steps, Hypothesis automatically searches for the shortest sequence that still fails, often 3 or 4 steps, and prints it.
- **Fuzzer:** the child writer prints each key only after `put` returns, flushing stdout, so whatever the parent reads is exactly the acknowledged set. `child.kill()` sends `SIGKILL`; data already written to the pipe survives the kill.

Interview angle: model-based testing checks the whole engine against a trivially correct reference, which finds bugs no hand-written test thinks of. The fuzzer writes each key only once per iteration; otherwise a later, unacknowledged write of the same key could legitimately be on disk and cause a false failure.

Supporting pieces (P0 to P2)

11. [C++] Varint encoding (P1)

```
void PutVarint32(std::string* dst, uint32_t v) {
  while (v >= 0x80) {                                        // more than 7
bits left
    dst->push_back(static_cast<char>((v & 0x7F) | 0x80));    // low 7 bits +
"more follows" flag
    v >>= 7;
  }
  dst->push_back(static_cast<char>(v));                      // last byte:
flag bit clear
}

bool GetVarint32(std::string_view* in, uint32_t* v) {
  uint32_t result = 0;
  for (int shift = 0; shift <= 28 && !in->empty(); shift += 7) {
    const uint8_t byte = static_cast<uint8_t>((*in)[0]);
    in->remove_prefix(1);
    result |= static_cast<uint32_t>(byte & 0x7F) << shift;
    if ((byte & 0x80) == 0) { *v = result; return true; }
  }
  return false;                                              // truncated or
too long: corrupt
}
```

Line by line:

- `0x80` is 128, binary `1000 0000`; `0x7F` is 127, binary `0111 1111`. Each output byte holds 7 bits of the number in its low bits; the top bit says whether another byte follows.
- `v & 0x7F`: keep the lowest 7 bits. `| 0x80`: set the "more follows" bit. `v >>= 7`: drop the 7 bits just written.
- Example: 300 is binary `1 0010 1100`. First byte: low 7 bits `010 1100` plus the flag → `0xAC`. Remaining value 2 → second byte `0x02`. So 300 takes 2 bytes instead of 4.
- `std::string_view* in`: a pointer to a view, so the function can advance the caller's view past the bytes it consumed (`remove_prefix`), like a cursor.
- `shift <= 28`: a 32-bit number needs at most 5 bytes; anything longer is corrupt data, not a valid varint.
- `return false` on malformed input: never trust bytes read from disk.

Python equivalent:

```
def put_varint(n: int) -> bytes:
    out = bytearray()
    while n >= 0x80:
        out.append((n & 0x7F) | 0x80)
        n >>= 7
    out.append(n)
    return bytes(out)
```

Interview angle: varints are how Protocol Buffers, LevelDB, and SQLite save space on lengths, which are usually small. The decoder's bounds checks show you treat on-disk data as untrusted input.

12. [C++] fsync, done correctly on every OS (P0)

```
Status PosixWritableFile::Sync() {
#if defined(__APPLE__)
  // macOS fsync() stops at the drive's cache; F_FULLFSYNC flushes the drive
itself.
  if (::fcntl(fd_, F_FULLFSYNC) == 0) return Status::OK();
  // Some filesystems don't support it; fall back to plain fsync.
#endif
  if (::fsync(fd_) != 0) {
    return Status::IOError(path_, std::strerror(errno));     // treat as
fatal
  }
  return Status::OK();
}
```

Line by line:

- `#if defined(__APPLE__) ... #endif`: **preprocessor** directives that include code only when compiling on macOS. Python's closest equivalent is `if sys.platform == "darwin":`, except this decision happens at compile time.
- `::fcntl(fd_, F_FULLFSYNC)`: the leading `::` means "the global C function," here the operating system's file-control call. `fd_` is the file descriptor, the OS's integer handle for an open file (Python: `f.fileno()`).
- `::fsync(fd_)`: the POSIX call that flushes the file from the OS page cache to the drive.

- `std::strerror(errno)`: turns the OS error number into readable text, like Python's `OSError.strerror`.

Python equivalent: `os.fsync(fd)`, or on macOS `fcntl.fcntl(fd, fcntl.F_FULLFSYNC)`.

Interview angle, a deep cut: if `fsync` fails, you cannot simply retry. On Linux the kernel may have already dropped the dirty pages and cleared the error, so a second `fsync` can "succeed" while data is lost. Postgres hit this in 2018 ("fsyncgate") and now crashes on `fsync` failure. StrataKV should treat it as fatal too.

13. [C++] Looking up a key in one SSTable (P2)

```
// found == false means "not in this file, keep searching older files".
Status SSTableReader::Get(std::string_view lookup_key, std::string_view
user_key,
                          std::string* value, bool* found) {
  if (!filter_.MayContain(user_key)) { *found = false; return Status::OK(); }
// zero disk reads

  auto it = std::lower_bound(index_.begin(), index_.end(), lookup_key,
      [&](const IndexEntry& e, std::string_view k) { return
icmp_.Compare(e.last_key, k) < 0; });
  if (it == index_.end()) { *found = false; return Status::OK(); }

  std::string block;
  if (auto s = file_->ReadAt(it->offset, it->size, &block); !s.ok()) return
s;  // one disk read

  for (BlockIter b(block); b.Valid(); b.Next()) {
    if (icmp_.Compare(b.key(), lookup_key) >= 0) {
      if (UserKey(b.key()) != user_key) break;          // passed where the
key would be
      *found = true;
      if (TypeOf(b.key()) == kTypeDeletion) return Status::NotFound();
      value->assign(b.value());
      return Status::OK();
    }
  }
  *found = false;
  return Status::OK();
}
```

Line by line:

- `bool* found`: a second output, separating "this file doesn't have the key, keep looking" from "this file says it's deleted, stop." Both would otherwise look like "no value."
- `filter_.MayContain(user_key)`: the bloom filter check. For most absent keys the function returns here without touching disk.
- `std::lower_bound(begin, end, value, comparator)`: binary search for the first index entry whose last key is not less than the lookup key. That data block is the only one that could hold the key. Python: `bisect.bisect_left`.
- `it == index_.end()`: the key is larger than everything in the file.
- `file_->ReadAt(...)`: the single disk read, about 4 KB.
- The `for` loop: scan the block's sorted entries until reaching the lookup key's position.
- `UserKey(b.key()) != user_key`: we reached the right position but it holds a different key, so this file doesn't have it.
- `kTypeDeletion`: a tombstone here means the key was deleted, so return `NotFound` and stop searching older files.

Python equivalent of the index search:

```
i = bisect.bisect_left(index_last_keys, lookup_key)   # first block whose
last key >= lookup_key
if i == len(index_last_keys):
    return NOT_IN_FILE
block = f.pread(index[i].size, index[i].offset)
```

Interview angle: a point lookup costs at most one disk read per file whose bloom filter says "maybe," plus the in-memory work. Without filters, a miss would read one block from every L0 file and the L1 file.

14. [C++] Crash points for testing (P1)

```
void CrashPoint(const char* name) {
  static const char* armed = std::getenv("STRATAKV_CRASH_AT");  // read once,
on first call
  if (armed != nullptr && std::strcmp(armed, name) == 0) {
    std::_Exit(1);  // die instantly: no destructors, no buffer flushing,
like SIGKILL
  }
}

// Inside the flush code:
WriteSSTable(imm, &meta);
CrashPoint("flush_after_sst_write");
versions_->LogAndApply(edit);          // MANIFEST now lists the new file
CrashPoint("flush_before_wal_delete");
env_->DeleteFile(old_wal);
```

Line by line:

- `const char* name`: a C-style string, a pointer to characters ending in a zero byte. Used here because it's cheap and these names are fixed literals.
- `static const char* armed = ...`: a **static local** is initialized once, the first time the function runs, and kept for all later calls. C++ guarantees this is thread-safe.
- `std::getenv`: reads an environment variable, like `os.environ.get`.
- `std::strcmp(a, b) == 0`: C-style string equality.
- `std::_Exit(1)`: ends the process immediately. Unlike `exit()`, it skips destructors and buffer flushes, imitating a real crash. Python: `os._exit(1)`.
- **Placement is the point:** each crash point sits between two durability steps. The test suite arms each one in turn and checks that recovery handles that exact moment.

Interview angle: random `SIGKILL`s find bugs probabilistically; named crash points hit the dangerous windows deterministically. Using both is the strongest answer to "how do you know recovery works?"

15. [C++] WriteBatch: atomic groups of writes (P1)

```
class WriteBatch {
 public:
  void Put(std::string_view key, std::string_view value) {
    rep_.push_back(static_cast<char>(kTypeValue));
    PutLengthPrefixed(&rep_, key);       // varint length, then the bytes
    PutLengthPrefixed(&rep_, value);
    ++count_;
  }
  void Delete(std::string_view key) {
    rep_.push_back(static_cast<char>(kTypeDeletion));
    PutLengthPrefixed(&rep_, key);
    ++count_;
  }
  std::string_view Contents() const { return rep_; }

 private:
  std::string rep_;      // encoded operations: becomes one WAL record's
payload
  uint32_t count_ = 0;
};
```

Line by line:

- `class WriteBatch { public: ... private: ... };` : `public` members are usable by anyone; `private` ones only inside the class. Python only has the `_name` convention.
- `std::string rep_` : used as a growable byte buffer, not text. C++ strings can hold any bytes, including zeros.
- `push_back` , `PutLengthPrefixed` : append the operation type, then each key and value with its length in front (snippet 11), so a reader knows where each ends.
- `++count_` : number of operations, written into the WAL record so replay knows how many to decode.
- `std::string_view Contents() const` : returns a view of the buffer without copying; `const` promises the method doesn't change the batch.
- `uint32_t count_ = 0;` : a default member value, like a dataclass field default.

Python equivalent:

```
class WriteBatch:
    def __init__(self):
        self.rep, self.count = bytearray(), 0
    def put(self, key: bytes, value: bytes):
        self.rep += bytes([TYPE_VALUE]) + put_varint(len(key)) + key +
put_varint(len(value)) + value
        self.count += 1
```

Interview angle: the whole batch becomes one WAL record with one checksum, so after a crash it replays entirely or not at all. That's how StrataKV makes multi-key writes atomic without a transaction manager.